

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
по дисциплине
SDR

по теме:
Дискретная свертка. Реализация приема и передачи BPSK сигналов

Студент:
Группа ИА-331

К.А Любимов

Преподаватели:
Лектор
Практик
Практик

Калачиков А.А
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

1 ДИСКРЕТНАЯ СВЕРТКА. РЕАЛИЗАЦИЯ ПРИЕМА И ПЕРЕДАЧИ BPSK СИГНАЛОВ

I [Ссылка на GitHub](#)

Цель практики:

Программно реализовать дискретную свертку, используя язык программирования C++, передать полученные после свертки биты, посмотреть, что мы получим на выходе

Краткие теоретические сведения

Дискретная свертка — это математическая операция над двумя дискретными последовательностями (сигналами), определяемая следующим образом:

Пусть даны две последовательности $x[n]$ и $h[n]$, где $n \in \mathbb{Z}$. Их свертка $y[n]$ вычисляется по формуле:

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - k]$$

Для последовательностей конечной длины L и M , где $x[n]$ определена при $0 \leq n \leq L - 1$, а $h[n]$ — при $0 \leq n \leq M - 1$, результирующая последовательность $y[n]$ будет иметь длину $N = L + M - 1$ и формула принимает вид:

$$y[n] = \sum_{k=\max(0, n-M+1)}^{\min(n, L-1)} x[k] \cdot h[n - k], \quad \text{для } 0 \leq n \leq N - 1$$

Свойства дискретной свертки:

- **Коммутативность:** $x * h = h * x$
- **Ассоциативность:** $(x * h) * g = x * (h * g)$
- **Дистрибутивность:** $x * (h + g) = (x * h) + (x * g)$

Передавать мы будем биты после свертки. Перед этим биты нужно удлинить, за это будет отвечать функция `upsample`, которая из одного бита делает например 10. То есть у нас было `[1011]` битов, а стало(если `upsample` до 5) `[10000,00000,10000,10000]`, далее эти биты идут в функцию под названием `filter`, которая выполняет свертку с единичным импульсом после которой у нас получается такой результат `[11111,00000,11111,11111]`. Получается, что мы продлеваем наш сигнал.

BPSK и QPSK

BPSK (Binary Phase Shift Keying) — это цифровая модуляция, при которой двоичные символы 0 и 1 передаются изменением фазы несущего сигнала на 0 и π радиан соответственно. Сигнал BPSK можно записать как:

$$s_{\text{BPSK}}(t) = A \cos(2\pi f_c t + \phi), \quad \phi = \begin{cases} 0, & \text{для символа 1,} \\ \pi, & \text{для символа 0.} \end{cases}$$

QPSK (Quadrature Phase Shift Keying) — это модуляция, передающая два бита за символ с использованием четырёх фазовых состояний, разнесённых на $\frac{\pi}{2}$ радиан. Сигнал QPSK имеет вид:

$$s_{\text{QPSK}}(t) = A \cos\left(2\pi f_c t + \frac{(2k+1)\pi}{4}\right), \quad k = 0, 1, 2, 3,$$

что соответствует фазам $\frac{\pi}{4}, \frac{3\pi}{4}, \frac{5\pi}{4}, \frac{7\pi}{4}$.

BPSK обеспечивает более высокую помехоустойчивость, тогда как QPSK удваивает спектральную эффективность по сравнению с BPSK при том же энергетическом расходе на бит.

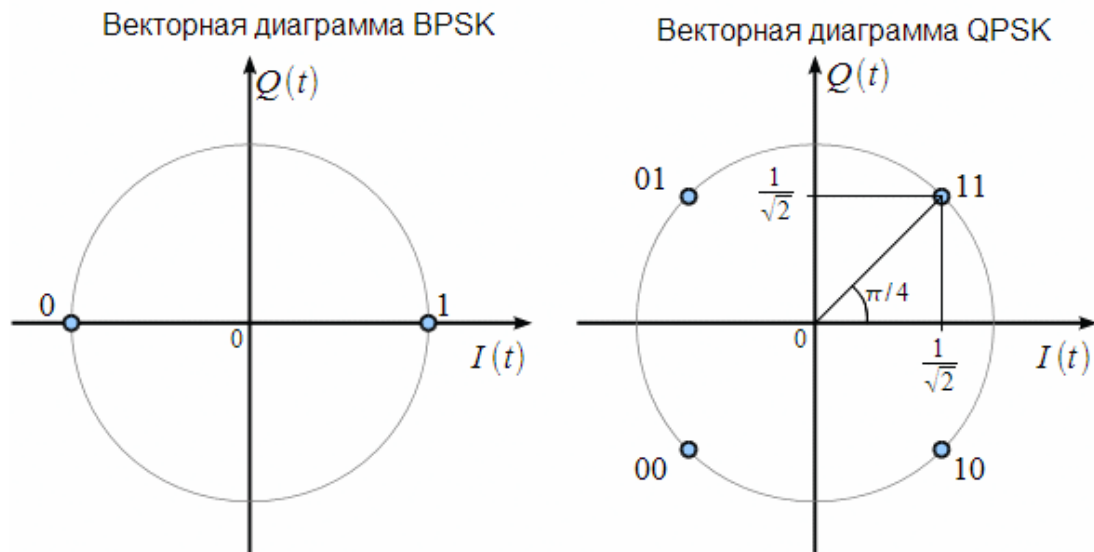


Рис. 1 — Векторная диаграмма BPSK, QPSK

На осциллографе QPSK сигнала, должно быть видно как меняется фаза

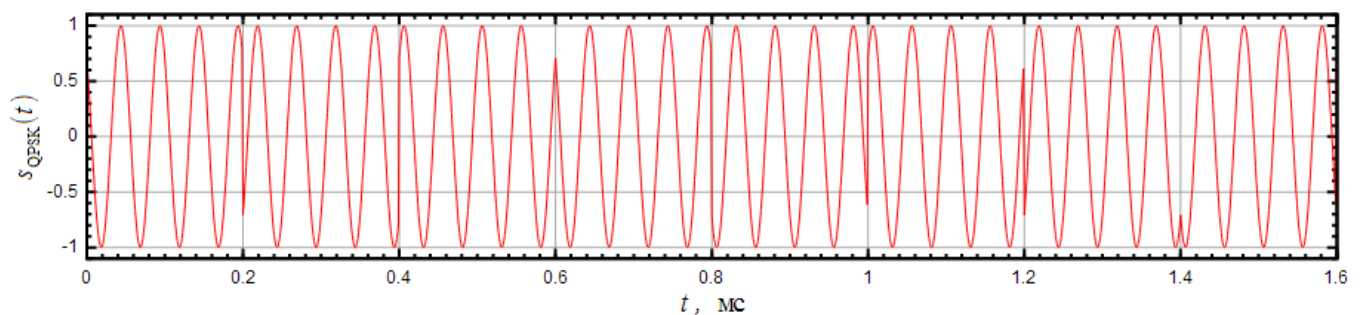


Рис. 2 — Осциллограмма QPSK сигнала

Структурная схема QPSK модулятора будет выглядеть так:

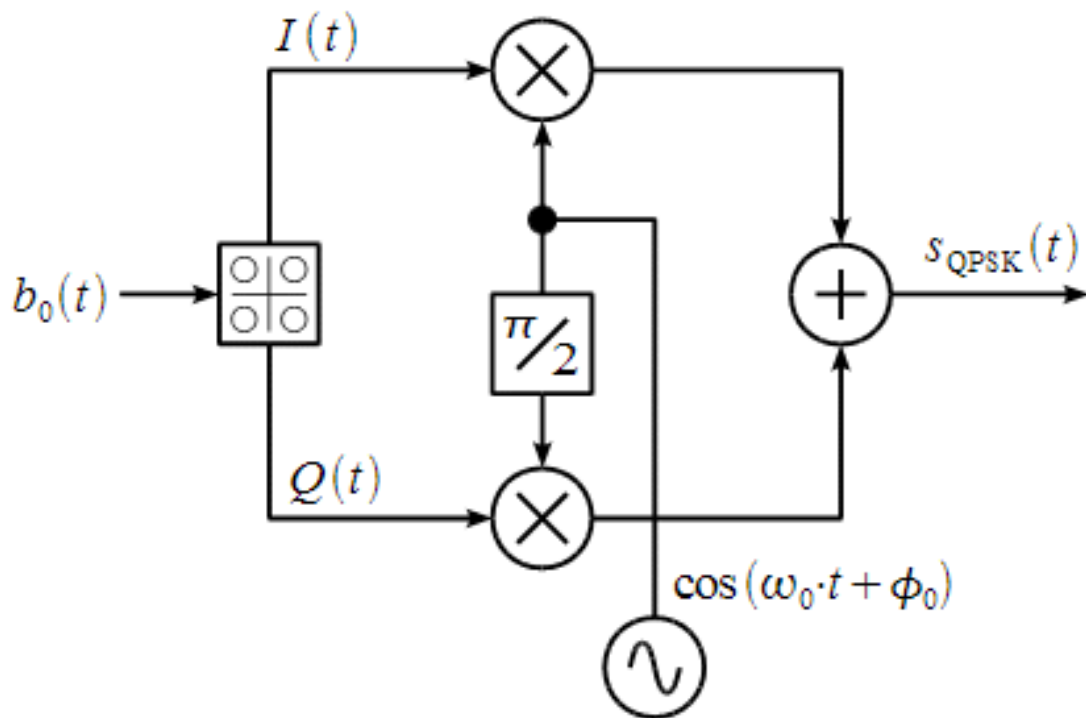


Рис. 3 — Структурная схема QPSK модулятора

Выполнение

Вместо BPSK, будем использовать QPSK. Напишем код для перевода в QPSK символы

Листинг 1.1 — map для QPSK

```

1 // QPSK СИМВОЛЫ
2 const complex<double> i0 (0, 0);
3 const complex<double> i1 (1, 1);
4 const complex<double> i2 (-1, 1);
5 const complex<double> i3 (-1, -1);
6 const complex<double> i4 (1, -1);
7
8 static map<string, complex<double>> qpsk_map = {
9     {"00", i1},
10    {"01", i2},
11    {"11", i3},
12    {"10", i4}
13 };

```

Функция для UpSampler, которая удлиняет сигнал, реализована с помощью простых циклов

Листинг 1.2 — UpSampler

```

1 void UpSampler(complex<double> *symbols, int len_s, complex<double> *symbols_ups, int
   L){
2     for (size_t i = 0; i < (size_t)len_s*(size_t)L; i++) symbols_ups[i] = i0;
3     for (size_t i = 0; i < (size_t)len_s; i++) symbols_ups[i*L] = symbols[i];
4 }

```

Функция для свертки, называется в моем коде filter, реализует процесс свертки массива уже после upsample с еденичным импульсом с заданной нами длиной

Листинг 1.3 — filter

```

1 void filter(complex<double> *symbols_ups, int len_symbols_ups, complex<double> *
  impulse, int L) {
2 vector<complex<double>> sum(len_symbols_ups, 0);
3 for (size_t i = 0; i < (size_t)len_symbols_ups; i++) {
4     for (size_t j = 0; j < (size_t)L && (int)(i-j) >= 0; j++) {
5         sum[i] += impulse[j] * symbols_ups[i-j];
6     }
7 }
8 memcpy(symbols_ups, sum.data(), len_symbols_ups * sizeof(complex<double>));
9 }

```

Функция Mapper, которая формирует из 2 битов один символ, который далее соотносится к соответствующему комплексному значению в нашем qpsk_map.

Листинг 1.4 — Mapper

```

1 void Mapper(int16_t *bits, int len_b, complex<double> *symbols){
2     string pair_bits;
3     for (size_t i = 0; i < (size_t)len_b; i += 2){
4         pair_bits = to_string(bits[i]) + to_string(bits[i+1]);
5         symbols[i/2] = qpsk_map[pair_bits];
6     }
7 }

```

Для практики я случайно формирую 20 битов от 0 до 1, затем пропускаю их через Mapper для того чтобы сформировать из битов символы по 2 бита каждый, далее пропускаю эти символы через UpSampler чтобы "продлить" сигнал, затем подаю уже удлинненные символы на filter, который делает свертку с еденичным импульсом

Листинг 1.5 — Как это устроено в коде

```

1 int n = 20;
2 int16_t *bits = (int16_t*)malloc(n * sizeof(int16_t));
3 for (int i = 0; i < n; i++) bits[i] = rand() % 2;
4
5 int len_symbols = n / 2;
6 complex<double> *symbols = (complex<double>*)malloc(len_symbols * sizeof(complex<
  double>));
7 int L = 10;
8 int len_symbols_ups = len_symbols * L;
9 complex<double> *symbols_ups = (complex<double>*)malloc(len_symbols_ups * sizeof(
  complex<double>));
10 complex<double> impulse[L];
11 for (int i = 0; i < L; i++) impulse[i] = 1.0;
12
13 Mapper(bits, n, symbols);
14 UpSampler(symbols, len_symbols, symbols_ups, L);
15 filter(symbols_ups, len_symbols_ups, impulse, L);
16 // Show_ArrayСимволы("", symbols_ups, len_symbols_ups);
17
18 int16_t *tx_samples = (int16_t*)malloc(2 * config.tx_mtu * sizeof(int16_t));
19 for (size_t i = 0; i < (size_t)config.tx_mtu; i++) {
20     tx_samples[2*i] = (int16_t)((real(symbols_ups[i])) * 1000) << 4;
21     tx_samples[2*i + 1] = (int16_t)((imag(symbols_ups[i])) * 1000) << 4;
22 }
23
24 // memcpy(config.tx_buff, tx_samples, 2 * len_symbols_ups * sizeof(int16_t));
25 fwrite(tx_samples, sizeof(int16_t), 2 * len_symbols_ups, tx);

```

Так же для того чтобы увидеть наши биты нам нужен код на python, который будет визуализировать все, что мы получили

Листинг 1.6 — Визуализация полученных битов

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 print("""Choose 0 - Tx
5 Choose 1 - Rx""")
6
7 answer = int(input())
8
9 if answer == 1:
10     nx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np
11         .int16)
12 else:
13     nx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/tx.pcm", dtype=np
14         .int16)
15 rx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
16     int16)
17 tx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/tx.pcm", dtype=np.
18     int16)
19 print(f"RX: {rx}\nTX: {tx}")
20
21 samples = []
22
23 for x in range(0, len(nx), 2):
24     samples.append(nx[x] + 1j * nx[x+1])
25
26 ampl = np.abs(samples)
27 phase = np.angle(samples)
28 time = np.arange(len(samples))
29
30 # plt.subplot(3,1,1)
31 # plt.plot(time, ampl)
32 # plt.grid(True)
33 # plt.tight_layout()
34 # plt.title("Amplitude")
35
36 # plt.subplot(3,1,2)
37 # plt.plot(time, phase)
38 # plt.grid(True)
39 # plt.tight_layout()
40 # plt.title("Phase")
41
42 # plt.subplot(3,1,3)
43 plt.plot(time, nx[0::2])
44 # plt.plot(time, nx[1::2])
45 plt.grid(True)
46 plt.tight_layout()
47 plt.title("Real and Imag parts")
48 plt.show()

```

По итогу мы можем считать наши pcm файлы, в которых содержатся наши сгенерированные биты от 0 до 1, можем вывести их амплитуду, фазу и взять реальную и мнимую части

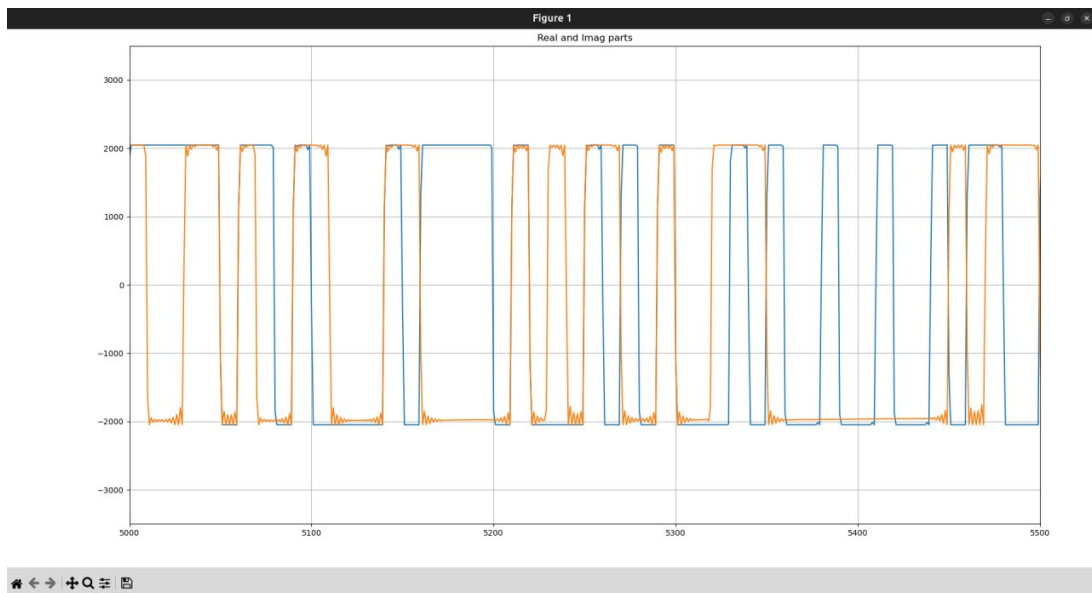


Рис. 4 — Реальная и мнимая части полученных битов

Вывод

Получилось отправить и принять, сгенерированные биты, которые прошли через Mapper, UpSampler, filter, отобразить их амплитуду, фазу, реальную и мнимую части, получить графики.